

Statistics Advance-4 — Assignment Answers

Data Science Masters | Google Colab / Jupyter Notebook Ready

Q1. Difference between t-test and z-test.

A z-test is generally used when the population standard deviation is known or the sample is large under the usual assumptions. A t-test is generally used when the population standard deviation is unknown and is estimated from the sample. Example: use a t-test to test a small sample mean when population standard deviation is unknown; use a z-test when the population standard deviation is known.

```
# t-test example
from scipy.stats import ttest_1samp
sample = [12, 14, 15, 13, 16]
print(ttest_1samp(sample, popmean=10))

# z-test example
from scipy.stats import norm
xbar, mu, sigma, n = 52, 50, 8, 64
z = (xbar - mu) / (sigma / (n ** 0.5))
print("z-score:", z)
```

Q2. Differentiate between one-tailed and two-tailed tests.

A one-tailed test checks for an effect in one specified direction (greater than or less than the hypothesized value). A two-tailed test checks for a difference in either direction. At a 5% significance level, a two-tailed test places 2.5% in each tail, while a one-tailed test places 5% in one tail.

```
# One-tailed alternative
# H1: mu > mu0

# Two-tailed alternative
# H1: mu != mu0
```

Q3. Explain Type 1 and Type 2 errors with examples.

Type I error means rejecting a true null hypothesis (false positive). Type II error means failing to reject a false null hypothesis (false negative). Example: if H0 says a drug has no effect, declaring that it works when it actually does not is Type I; failing to detect a real effect is Type II.

```
alpha = 0.05 # probability threshold commonly used for Type I error
beta = 0.20 # example Type II error probability
print("Alpha:", alpha)
print("Beta:", beta)
```

Q4. Explain Bayes' theorem with an example.

Bayes' theorem updates the probability of an event after observing evidence: $P(A|B) = P(B|A)P(A) / P(B)$. Example: suppose $P(D)=0.01$, $P(\text{Positive}|D)=0.95$, and $P(\text{Positive}|\text{not } D)=0.05$. Then $P(D|\text{Positive}) = 0.01*0.95 / (0.01*0.95 + 0.99*0.05) \approx 0.161$. So the posterior probability is about 16.1%.

```
P_D = 0.01
P_pos_given_D = 0.95
P_pos_given_not_D = 0.05

P_pos = P_pos_given_D*P_D + P_pos_given_not_D*(1-P_D)
posterior = (P_pos_given_D*P_D) / P_pos

print("P(D | Positive) =", posterior)
```

Q5. What is a confidence interval? How is it calculated?

A confidence interval gives a range of plausible values for a population parameter at a stated confidence level. For a mean with known population standard deviation, $CI = \bar{x} \pm z^* (\sigma/\sqrt{n})$. If σ is unknown, a t critical value is normally

used: $\bar{x} \pm t^* (s/\sqrt{n})$.

```
import scipy.stats as stats
import math

mean = 50
std = 5
n = 100
confidence = 0.95

z = stats.norm.ppf(1 - (1-confidence)/2)
margin = z * std / math.sqrt(n)
ci = (mean - margin, mean + margin)

print("95% CI:", ci)
```

Q6. Use Bayes' theorem for a sample problem and solution.

Suppose 2% of items are defective. A test detects a defective item 90% of the time and gives a positive result for 5% of non-defective items. The probability that an item is actually defective given a positive result is $0.02*0.90 / (0.02*0.90 + 0.98*0.05) \approx 26.92\%$.

```
P_D = 0.02
P_pos_D = 0.90
P_pos_notD = 0.05

P_pos = P_pos_D*P_D + P_pos_notD*(1-P_D)
P_D_pos = P_pos_D*P_D / P_pos

print("P(Defective | Positive) =", P_D_pos)
print("Percentage =", P_D_pos * 100)
```

Q7. Calculate the 95% confidence interval for mean 50 and standard deviation 5.

The assignment gives the mean and standard deviation but does not give the sample size. A numerical confidence interval therefore cannot be uniquely calculated. If 5 is the population standard deviation, the formula is $50 \pm 1.96*(5/\sqrt{n})$. The sample size n is required for the final numerical interval.

```
import math

mean = 50
std = 5
n = 100 # Example only; replace with the actual sample size

margin = 1.96 * std / math.sqrt(n)
print("95% CI:", (mean - margin, mean + margin))
# With n=100, CI = (49.02, 50.98)
```

Q8. What is margin of error? How does sample size affect it?

Margin of error is the amount added to and subtracted from the sample estimate to form a confidence interval. For a mean with known σ , $ME = z*(\sigma/\sqrt{n})$. As sample size increases, $\sqrt{n}$ increases, so the margin of error decreases. For example, increasing n from 100 to 400 halves the margin of error.

```
import math

z = 1.96
sigma = 10

for n in [100, 400]:
    me = z * sigma / math.sqrt(n)
    print("n =", n, "Margin of error =", me)
```

Q9. Calculate and interpret the z-score.

For $x=75$, population mean $\mu=70$, and population standard deviation $\sigma=5$: $z=(75-70)/5=1$. The observation is one standard deviation above the population mean.

```
x = 75
mu = 70
```

```
sigma = 5

z = (x - mu) / sigma
print("z-score:", z)
```

Q10. Test whether the weight-loss drug is significantly effective at 95% using a t-test.

Using $H_0: \mu = 0$ pounds lost and $H_1: \mu > 0$, with $n=50$, sample mean=6, and sample SD=2.5: $t = (6-0)/(2.5/\sqrt{50}) \approx 16.97$, $df=49$. This is far beyond the 5% one-tailed critical value, so reject H_0 . The sample provides very strong evidence of a positive average effect under this hypothesis.

```
from scipy.stats import t
import math

n = 50
sample_mean = 6
sample_std = 2.5
mu0 = 0

t_stat = (sample_mean - mu0) / (sample_std / math.sqrt(n))
df = n - 1
p_value = t.sf(t_stat, df)

print("t-statistic:", t_stat)
print("df:", df)
print("one-tailed p-value:", p_value)
print("Significant at 5%:", p_value < 0.05)
```

Q11. 95% confidence interval for a proportion of 65% from n=500.

For $p=0.65$ and $n=500$, the 95% normal-approximation CI is $p \pm 1.96\sqrt{p(1-p)/n}$. This gives approximately (0.608, 0.692), or 60.8% to 69.2%.

```
import math

p = 0.65
n = 500
z = 1.96

se = math.sqrt(p * (1-p) / n)
margin = z * se
ci = (p - margin, p + margin)

print("95% CI:", ci)
print("Percentage CI:", ci[0]*100, "% to", ci[1]*100, "%")
```

Q12. Compare two teaching methods using a t-test at significance level 0.01.

The assignment provides means (85 and 82) and standard deviations (6 and 5), but it does not provide sample sizes. A two-sample t-test cannot be numerically completed without the sample sizes (or equivalent standard errors). Therefore, the source information is insufficient for a unique t-statistic and p-value.

```
from scipy.stats import ttest_ind
import numpy as np

# Replace these arrays with the actual observations from Method A and B.
method_A = np.array([85]) # placeholder
method_B = np.array([82]) # placeholder

# ttest_ind requires actual observations (or another method using
# sample sizes, means, and standard deviations).
# result = ttest_ind(method_A, method_B, equal_var=False)
# print(result)
```

Q13. Calculate the 90% confidence interval for population mean.

Given population mean=60, population SD=8, $n=50$, and sample mean=65. Since the population standard deviation is given, use $z=1.645$ for a 90% CI: $65 \pm 1.645*(8/\sqrt{50}) \approx 65 \pm 1.861$. CI $\approx$ (63.14, 66.86).

```
import math
sample_mean = 65
```

```

population_std = 8
n = 50
z = 1.645

margin = z * population_std / math.sqrt(n)
ci = (sample_mean - margin, sample_mean + margin)

print("90% CI:", ci)

```

Q14. Test whether caffeine has a significant effect on reaction time at 90% using a t-test.

The assignment gives $n=30$, $\text{mean}=0.25$ seconds, and $\text{SD}=0.05$ seconds, but it does not specify the hypothesized/reference mean for reaction time. Without that null value, the t-statistic and p-value cannot be uniquely calculated. If the intended H_0 value is μ_0 , the one- or two-tailed t-test can be completed using the formula below.

```

from scipy.stats import t
import math

n = 30
sample_mean = 0.25
sample_std = 0.05

mu0 = 0.25 # Replace with the reference mean specified by the study
t_stat = (sample_mean - mu0) / (sample_std / math.sqrt(n))
df = n - 1

# For a two-tailed 90% test:
p_value = 2 * t.sf(abs(t_stat), df)

print("t-statistic:", t_stat)
print("df:", df)
print("two-tailed p-value:", p_value)

```